

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA1304

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques.*

(BL : 3)

Assessment Tool Weightage: 10%

QUESTIONS: 20 Total Marks:20 Duration :1 Hour

Mock Interview question

Code of Conduct:

I Girija B 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student: GIRIJA B

SIMATS ENGINEERING ASSESSMENT TOOLS

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?

Pushdown Automaton (PDA): A PDA is an automaton that uses a **stack** along with input symbols to recognize **Context-Free Languages (CFLs)**.

Difference from Finite Automaton (FA):

- **FA:** Has only finite states and **no stack**.
- **PDA:** Has finite states **plus a stack** for storing symbols.
- **FA:** Recognizes **Regular Languages**.
- **PDA:** Recognizes **Context-Free Languages**.
- PDA is more powerful than FA because the stack provides additional memory.

2. Define the components of a PDA formally.

A **Pushdown Automaton (PDA)** is formally defined as a **7-tuple**:

$$M=(Q,\Sigma,\Gamma,\delta,q_0,Z_0,F)$$

Where:

- **Q** – Finite set of states.
- **Σ** – Input alphabet.
- **Γ** – Stack alphabet.
- **δ** – Transition function.
- **q_0** – Initial state.
- **Z_0** – Initial stack symbol.
- **F** – Set of accepting/final states.

3. What is a stack in PDA? Why is it important?

A **stack** in a PDA is a memory structure that follows **LIFO (Last In, First Out)**.

Importance:

- Stores symbols during computation.
- Allows the PDA to remember an arbitrary amount of information.
- Helps recognize **Context-Free Languages** such as $a^n b^n$.
- Supports **push, pop, and read** operations.

4. What is an Instantaneous Description (ID) in PDA?

An **Instantaneous Description (ID)** represents the current configuration of a PDA at any point during computation.

It is written as:

$$(q, w, \alpha)$$

where:

- **q** = current state
- **w** = remaining input string
- **α** = current stack contents

It shows the **state, unread input, and stack status** of the PDA.

5. What are the different types of moves in a PDA?

The main types of PDA moves are:

1. **Push move** – Reads an input symbol and pushes a symbol onto the stack.
2. **Pop move** – Removes the top symbol from the stack.
3. **No-stack-change move** – Reads an input symbol without changing the stack.
4. **ϵ -move** – Changes state or stack without consuming an input symbol.

6. What is acceptance by final state and empty stack?

Acceptance by Final State: A string is accepted when the PDA finishes reading the input and reaches a **final/accepting state**.

Acceptance by Empty Stack: A string is accepted when the PDA finishes reading the input and the **stack becomes empty**, regardless of the final state.

7. Define the language accepted by a PDA.

The **language accepted by a PDA** is the set of all strings that the PDA accepts.

$$L(P) = \{w \in \Sigma^* \mid P \text{ accepts } w\}$$

A PDA accepts a string either by **reaching a final state** or by **emptying its stack**, depending on the acceptance method used.

8. How does a PDA process a string using IDs?

A PDA processes a string using **Instantaneous Descriptions (IDs)** by:

1. Starting with the **initial state, input string, and initial stack symbol**.
2. Applying transitions to **read input and modify the stack**.
3. Producing a sequence of IDs showing each step.
4. The string is **accepted** if the PDA reaches a final state or an empty stack, depending on the acceptance method.

9. Differentiate between Deterministic PDA and Non-Deterministic PDA.

DPDA: Has **only one possible move** for each configuration.

NPDA: Can have **multiple possible moves** for the same configuration.

DPDA: Recognizes **Deterministic CFLs**.

NPDA: Recognizes **all Context-Free Languages**.

10. Why are PDAs equivalent to Context-Free Grammars (CFG)?

PDAs are equivalent to **Context-Free Grammars (CFGs)** because both recognize exactly the **Context-Free Languages (CFLs)**.

- Every CFG can be converted into a PDA.
- Every PDA can be converted into a CFG.

Thus:

Languages accepted by PDA = Languages generated by CFG

11. What types of languages are accepted by PDA? Give examples.

A PDA accepts Context-Free Languages (CFLs).

Examples:

- $L = \{a^n b^n \mid n \geq 0\}$
- Balanced parentheses: $\{w \mid w \text{ has balanced parentheses}\}$
- Palindromes: $\{ww^R \mid w \in \{a,b\}^*\}$

Thus, **PDA** $\rightarrow$ **Context-Free Languages**.

12. What is a Turing Machine? How is it more powerful than PDA?

A **Turing Machine (TM)** is a computational model with a **finite control**, an **infinite tape**, and a **read/write head**.

Why it is more powerful than PDA:

- PDA has a **stack** with limited access (LIFO).
- TM has an **unbounded tape** that can be **read, written, and moved in both directions**.
- PDA recognizes **Context-Free Languages**, while TM can recognize **Recursively Enumerable Languages**.

Thus, **TM** is more powerful than PDA.

13. Define the components of a Turing Machine.

A **Turing Machine (TM)** is formally defined as a **7-tuple**:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where:

- **Q** – Finite set of states.
- **Σ** – Input alphabet.
- **Γ** – Tape alphabet.
- **δ** – Transition function.
- **q_0** – Initial state.
- **B** – Blank symbol.
- **F** – Set of final/accepting states.

14. Explain tape, head, and state transitions in a Turing Machine.

Tape: An infinite memory divided into cells, used to store input and symbols.

Head: Reads and writes symbols on the tape and moves **left or right**.

State Transition: Determines the next state, symbol to write, and head movement based on the current state and tape symbol.

15. Explain programming techniques for Turing Machines.

Common techniques include:

- **Marking:** Replace symbols with marked versions to remember processed symbols.
- **Scanning:** Move the head left or right to search for specific symbols.
- **Copying:** Duplicate symbols from one tape section to another.

- **Shifting:** Move symbols left or right to create space or rearrange data.
- **State-based control:** Use states to represent different stages of computation.
- **Subroutines:** Divide complex tasks into smaller reusable operations.

16. What is storage in finite control in Turing Machines?

Storage in finite control means using the **states of a Turing Machine** to store a small, fixed amount of information during computation.

- It stores information temporarily in the **finite control unit**.
- The amount of information is **limited** by the number of states.
- It is useful for remembering symbols or the current stage of processing.

17. Compare FA, PDA, and TM based on memory, power, and language acceptance.

FA (Finite Automaton):

It has **no additional memory** and accepts **Regular Languages**. It is the least powerful among the three.

PDA (Pushdown Automaton):

It uses a **stack as memory** and accepts **Context-Free Languages**. It is more powerful than FA.

TM (Turing Machine):

It uses an **infinite tape as memory** and accepts **Recursively Enumerable Languages**. It is the most powerful.

Power order:

FA < PDA < TM

18. What is DFA?

DFA (Deterministic Finite Automaton) is a finite-state machine in which, for every state and input symbol, there is **exactly one defined transition**.

It is used to recognize **Regular Languages**.

19. Which language is accepted by a Turing Machine?

A **Turing Machine (TM)** accepts **Recursively Enumerable (RE) Languages**.

Example:

$L = \{a^n b^n c^n \mid n \geq 1\}$

Thus, Turing Machines can recognize languages that are beyond the capability of **Finite Automata and PDAs**.

20. How Turing machines are applied in real world applications?

Turing Machines are mainly used as a **theoretical model of computation** to understand how computers

solve problems.

Real-world applications include:

- Designing and analyzing algorithms.
- Modeling computer programs and processors.
- Studying **computability and decidability**.
- Understanding the limits of what computers can solve.
- Providing the theoretical foundation for modern computer science.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand